Basket Catch 6 - Audio

Sound can make a big difference in the feel of a game. Giving your player audio feedback when something happens can elevate the experience. Audio in Godot is fairly straight forward but requires a little bit of setup. Let's go through some best practices.

AudioStreamPlayer

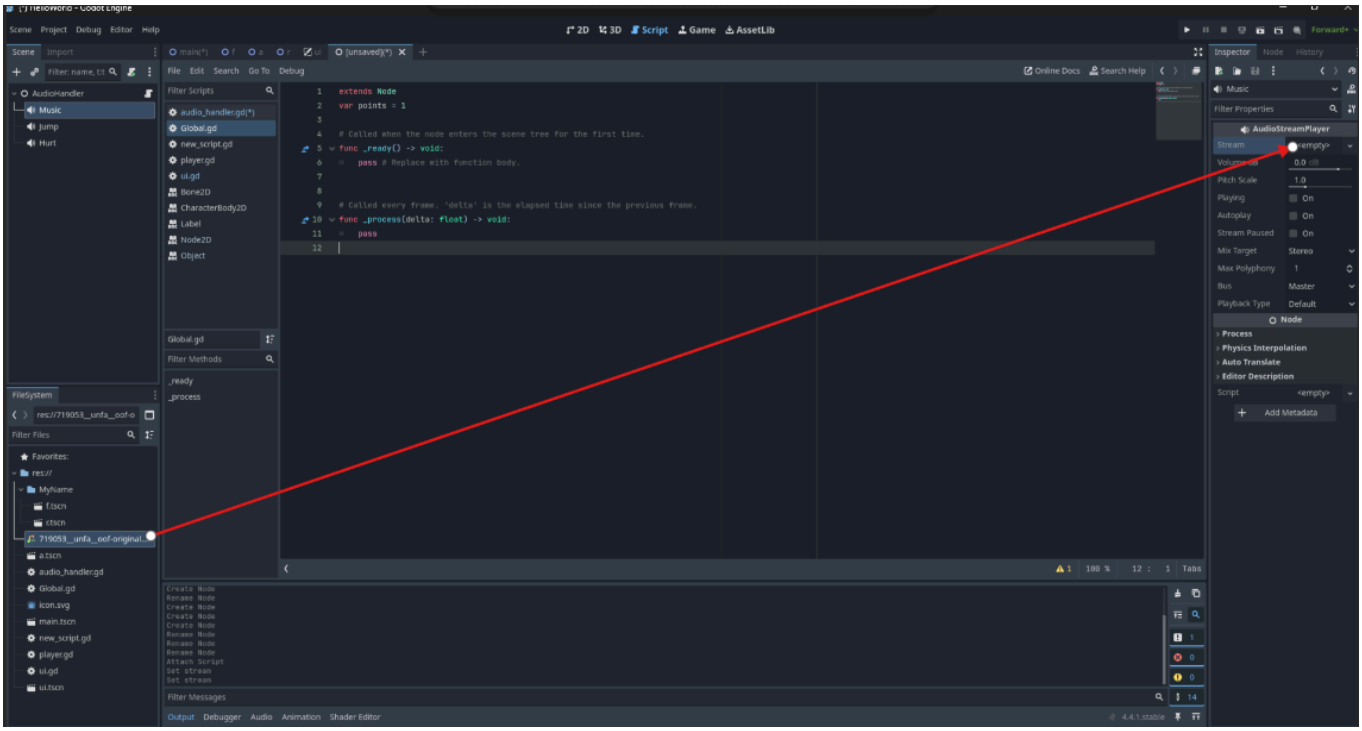
The AudioStreamPlayer is Godot's general purpose tool for playing sound. This Node cannot generate audio itself but rather takes a sound file (an .mp3, .ogg, .wav, etc. note that not all audio files are supported) and can then play it back by calling the .play() method. This means that you will need to create a separate AudioStreamPlayer for every sound you want in your game, including any music.

Setting up an AudioStreamPlayer

Our AudioStreamPlayer has many useful properties. Here's the most commonly used:

Property	What it does
stream	The audio file to play back
volume_db	The volume of the audio. 0 is the original volume, negative numbers quieter, and positive numbers louder.
pitch_scale	The pitch (and speed) of the playback. 0.5 is an octave down. 2 is an octave up. 4 is two octaves up. etc.
autoplay	Plays the AudioStreamPlayer when it appears
max_polyphony	The amount of overlapping sounds the AudioStreamPlayer can make

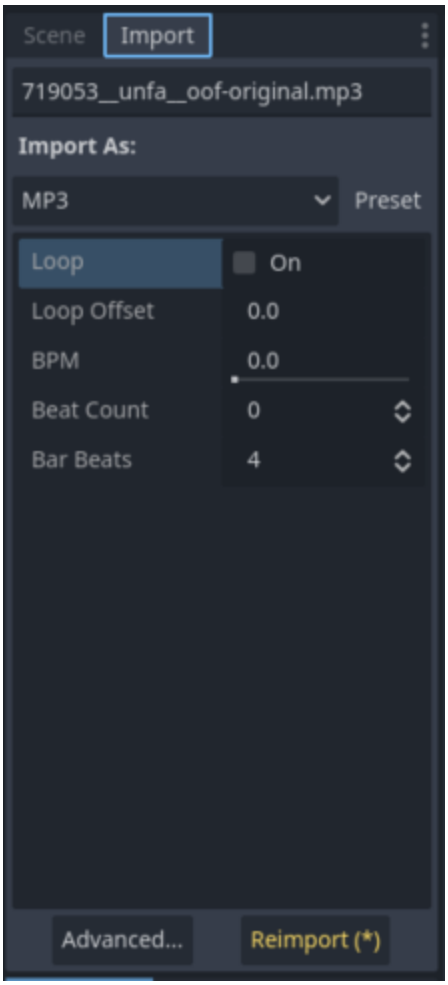
To add an audio file we will first import it by dragging the file from our computer's file browser and dropping it into Godot's FileSystem. We will then drag the sound file from the FileSystem to the AudioStreamPlayer's stream property in the inspector:



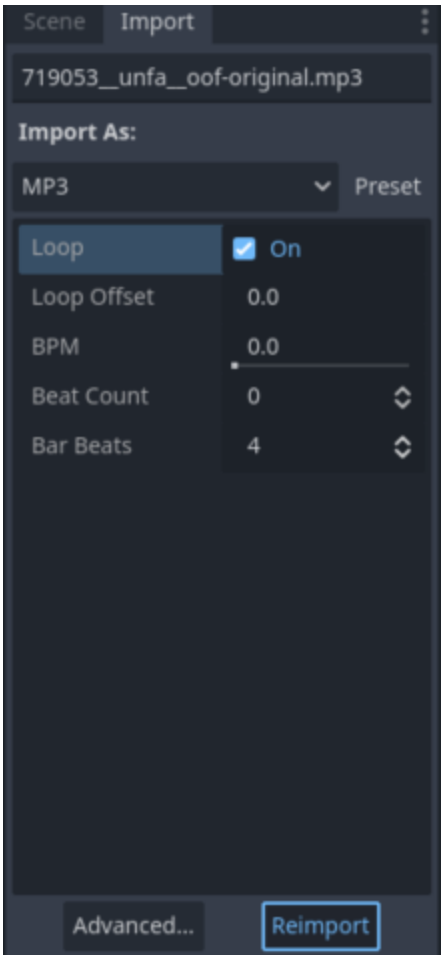
Then we may want to adjust some settings. We may want to reduce the volume of our sound file if it is meant to be background music. We will do this by making the volume_db property a *negative* number. volume_db is a boost/cut in *decibels*, meaning that we are adjusting the volume relative to its original volume. The decibel scale is a *logarithmic* scale, meaning that every 10 decibels is perceived as either half as loud (negative numbers) or twice as loud (positive numbers). This means that increasing a sounds volume_db by 10 doubles its volume, and increasing its volume_db by 20 *quadruples* its volume. It is safest not to set volume_db *above* zero though, only to lower the volume.

We may also want to be able to have sounds overlap. For instance, our player might fire a projectile but we do not want the firing sound to end if the player fires again before it is finished. We can allow the AudioStreamPlayer to overlap sounds by changing its max_polyphony property. max_polyphony controls how many sounds can be heard at once. If it is set to 2, if we were to play 3 sounds in rapid succession, the first sound would be stopped when we play the third sound.

You may also want your audio to loop. To do this we need to change a setting in the Import tab, which can be found in the SceneTree dock. First, select your sound file in the FileSystem. Then click on the Import tab:

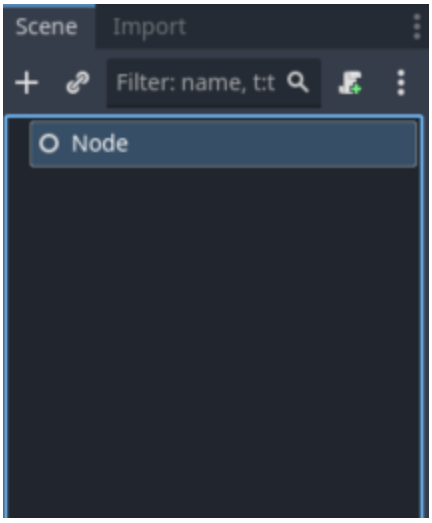


This will look slightly different for different audio files but all audio files should give an option for looping. We will need to enable looping here and the click "Reimport":

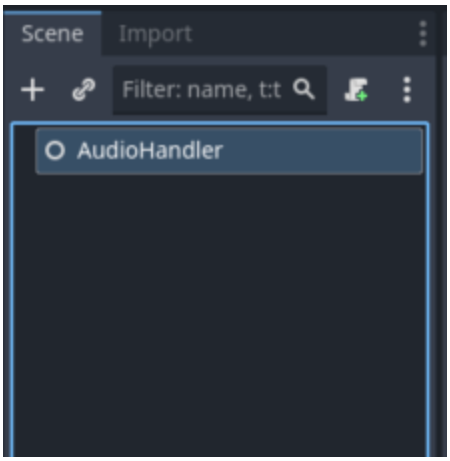


Organizing and Using Your AudioStreamPlayers

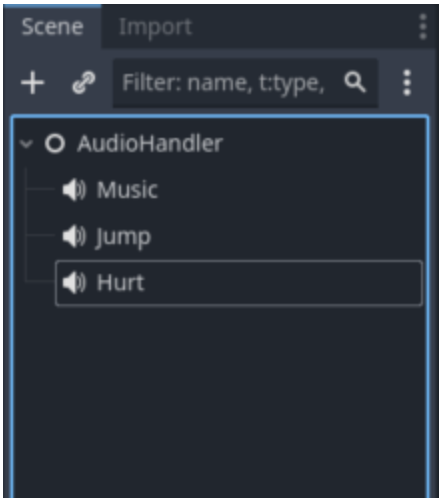
It is important to determine where in your game sounds might be heard. When should you be able to hear each sound? What should make the sound? How loud should the sound be? In general it is best for us to make a new scene to organize all of our AudioStreamPlayers in. This will look similar to how we organized and controlled our UI. First, we'll need to create a new scene by choosing "Other Node", after which we will choose the "Node" Node.



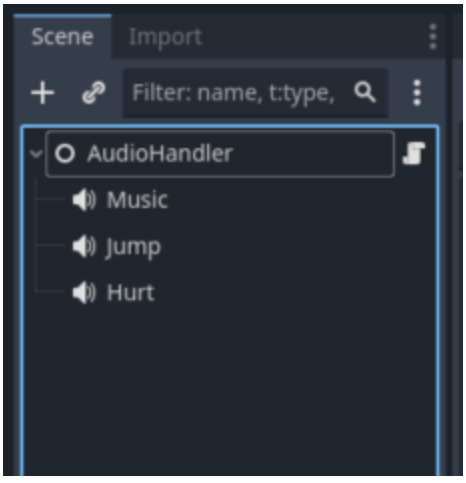
Then we'll rename it something like AudioHandler:



We can then add any of the sounds that we would like to play on this Node. Here I'll add three sounds: Background music, a jump sound, and a hurt sound. Remember to rename your AudioStreamPlayers as you add them:



Next, similar to how we worked with our UI, we will add a script to our root node. Here's what a properly formatted AudioHandler would look like:



Next we need to get references to our AudioStreamPlayers into our script. We can do this by dragging them into the script, holding ctrl, and releasing the mouse:

```
extends Node

@onready var music: AudioStreamPlayer = $Music
@onready var jump: AudioStreamPlayer = $Jump
@onready var hurt: AudioStreamPlayer = $Hurt



# Called when the node enters the scene tree for the first time.
func _ready() -> void:
    pass # Replace with function body.


# Called every frame. 'delta' is the elapsed time since the previous frame.
func _process(delta: float) -> void:
    pass
```

We're nearly finished with our setup, the last thing we need to do is add a way to *reference* this AudioHandler from a Global (also called an Autoload) script. We'll do this by first creating a new variable in our Global script:

```
var audio_handler
```

Then, back in the AudioHandler script, we will need to assign the AudioHandler to the Global.audio_handler property. We will do this in the AudioHandler's _ready() function:

```
extends Node

@onready var music: AudioStreamPlayer = $Music
@onready var jump: AudioStreamPlayer = $Jump
@onready var hurt: AudioStreamPlayer = $Hurt



# Called when the node enters the scene tree for the first time.
func _ready() -> void:
    Global.audio_handler = self
    pass # Replace with function body.


# Called every frame. 'delta' is the elapsed time since the previous frame.
func _process(delta: float) -> void:
```

```
    pass
```

We can then play any sound connected to the AudioHandler from *any* script by using the Global.audio_handler property. Here's an example of how we would play all three of these sounds:

```
Global.audio_handler.music.play()
Global.audio_handler.jump.play()
Global.audio_handler.hurt.play()
```

This process might seem complicated at first but there is a very good reason to work with audio in this way. For example, our hurt sound effect might play whenever we defeat an enemy. If we were to attach our "Hurt" AudioStreamPlayer to the enemy scene, and play it whenever it is defeated, that code might look like this:

```
@onready var hurt: AudioStreamPlayer = $Hurt

func on_defeat():
    hurt.play()
    queue_free()
```

This might look like it would work at first glance but we would immediately run into a problem. Here we remove the enemy scene and play our sound at the same time. However, our "Hurt" AudioStreamPlayer is *attached* to our enemy scene. This means that when the enemy scene is removed the "Hurt" AudioStreamPlayer is removed *as well*, stopping the sound from playing. By separating where the audio is played from the object making it we can avoid this unexpected behavior.

Now it's your turn!

In your Basket Catch project do the following:

1. Search online on a site like freesound.org for appropriate sounds for the following:
    1. A sound for catching your objects
    2. A sound for dropping an object
    3. Start screen music
    4. Background music
    5. End screen music
2. Make a new scene for handling your audio, add all of your sounds to this scene. (See the section "Organizing and Using Your AudioStreamPlayers" for properly setting this scene up)

3. Add this audio handler scene to your start scene, main scene, and end scene
4. Add an Area2D to your main scene that is below your screen. Attach a script to it. Using its body_entered() signal, delete any object that falls into it and play the "drop" sound
5. Play your "catch" sound whenever you catch an object
6. Play your Start screen music when your start screen appears (Make sure it loops)
7. Play your Background music when your main screen appears (Make sure it loops)
8. Play your End screen music when your end screen appears (Make sure it loops)